

3.5 Projection in Spring Data Jpa

• Database Projections in Spring Data JPA:

- **What is a Database Projection?**

- A database *projection* means: Fetching **only selected columns** from a database instead of the entire entity.

- **Why projections are needed:**

- Improve performance (less data fetched).
- Reduce memory usage.
- Avoid exposing full entities.
- Optimize read-only queries.

• Types of Projections in Spring Data JPA:

- Spring Data JPA supports **two** main DTO projection styles:
 - **Interface-based** Projections
 - **Class-based** (Constructor) **DTO** Projections

• Interface-Based Projections:

- An **interface-based projection** is a Java interface with **getter** methods that represent selected database columns.
- Spring Data JPA:
 - Executes the query.
 - Creates a **dynamic proxy**.
 - Returns a **read-only view** over query results.

• Basic Example:

- **Entity:**

```
@Entity
public class ProductEntity {
    private String title;
    private BigDecimal price;
}
```

- **Interface DTO:**

```
public interface ProductInfo {
    String getTitle();
    BigDecimal getPrice();
}
```

- **Repository:**

```
@Query("select e.title as title, e.price as price from ProductEntity e")
List<ProductInfo> findAllProducts();
```

- ✓ Only title and price are fetched
- ✓ No ProductEntity objects are created

3.5 Projection in Spring Data Jpa

• How Interface Projections Work Internally:

- **Spring Data JPA does NOT:**

- Create an entity
- Convert entity → DTO

- **Instead:**

1. Executes **JPQL**
2. Reads result set
3. Creates a **JDK dynamic proxy**
4. Getter calls are resolved from the query result

• Rules for Interface Projections:

- 1) Interface must contain **getters**
- 2) Getter **names** must **match**:
 - Entity field names **OR**
 - JPQL aliases
- 3) **Return types** must match **DB column types**
- 4) Interface projections are **read-only**
- 5) **Setters** are ignored
- 6) Validation is **NOT supported**
- 7) **Not suitable** for updates

• Class-Based DTO Projections:

- A **class-based projection** uses a DTO class with a **constructor**.
- Spring:
 - Executes JPQL
 - Instantiates DTO using constructor
 - Provides full type safety

• Example:

- **DTO:**

```
@Data
public class ProductInfoDTO {
    private final String title;
    private final BigDecimal price;
}
```

- **Repository:**

```
@Query("select new com.example.ProductInfoDTO(e.title, e.price) from ProductEntity e")
List<ProductInfoDTO> findAllProducts();
```

- ✓ Compile-time safety
- ✓ No proxy
- ✓ Clean behaviour

3.5 Projection in Spring Data Jpa

• Interface vs Class DTO (Comparison):

Feature	Interface Projection	Class DTO Projection
Object Creation	Dynamic proxy	Real object
Type Safety	✗ Weak	✓ Strong
Performance	✓ Slightly better	✓ Very good
Read-only	Yes	No
Setter Support	No	Yes
Validation	No	Yes
Risk of silent bugs	High	Low
Best Use Case	Simple read-only queries	APIs, business logic

• When to Use What:

• Use Interface Projections when:

- Query is **read-only**
- Only few columns are required
- **Performance** is **critical**
- You trust field names & types

• Use Class DTO Projections when:

- Data goes to **REST API**
- **Validation** is needed
- Mapping clarity is required
- You want strict **type safety**

• Aggregate Projection Queries in Spring Data JPA:

- Aggregate projection queries are used to **compute summary data** (COUNT, SUM, AVG, MIN, MAX) and map the results into **DTOs or projections** instead of full entities.
- This approach is efficient, read-only, and ideal for reporting and analytics.

• DTO-Based Aggregate Projection:

• Create a DTO:

- The DTO must expose a **public constructor** matching the query output.

@Data

```
public class CProductPriceType {  
    private final ProductType productType;  
    private final Long count;  
}
```

▸ Key rules:

- Constructor argument order must match the `select` clause
- Use **wrapper** types (Long, Double)
- DTOs are **not entities** and are **not managed**

3.5 Projection in Spring Data Jpa

- **Write the JPQL Query:**

- Use a constructor expression in JPQL.

```
@Query("""
    select new com.example.dtos.CProductPriceType(
        e.productType, COUNT(e)
    )
    from ProductEntity e
    group by e.productType
    order by COUNT(e) DESC
""")
```

List<CProductPriceType> **findAllProductTypeCount()**;

- Rules enforced by **JPA**:

- Every non-aggregated **field** in **select** must appear in **GROUP BY**
- **COUNT(entity)** always returns **Long**
- Ordering by aggregate functions is allowed

- **How It Works Internally:**

1. **JPQL** is **parsed** and **validated**
2. **JPQL** is **translated** into **SQL**
3. Database performs **grouping** and **aggregation**
4. Result rows are returned
5. Hibernate instantiates DTOs using the **constructor**

- **DML Queries in Spring Data JPA:**

- **DML** (Data Manipulation Language) queries are used to **modify database data**.

- In **Spring Data JPA**, DML operations are executed using **modifying queries**.

- **Core Rules for DML Queries:**

- **Rule 1:** Use **@Modifying**
 - Spring Data JPA assumes queries are **read-only** by default.
 - *@Modifying* tells Spring that the query **changes data**.
- **Rule 2:** Use **@Transactional**
 - DML queries must run inside a **transaction**.

- **When to Use DML Queries:**

- ✓ Use when:**

- Updating/deleting many rows
- Performance is critical
- No entity logic is needed

- ✗ Avoid when:**

- Entity lifecycle callbacks are required
- Business rules exist in entity methods
- You need automatic dirty checking